Serial communications interface (SCI)

What is a serial communications interface (SCI)?

A serial communications interface (SCI) is a device that enables the serial exchange of data -- that is, one bit at a time -- between a microprocessor and peripherals, such as printers, external drives, scanners and mice. In this respect, it is similar to a serial peripheral interface.

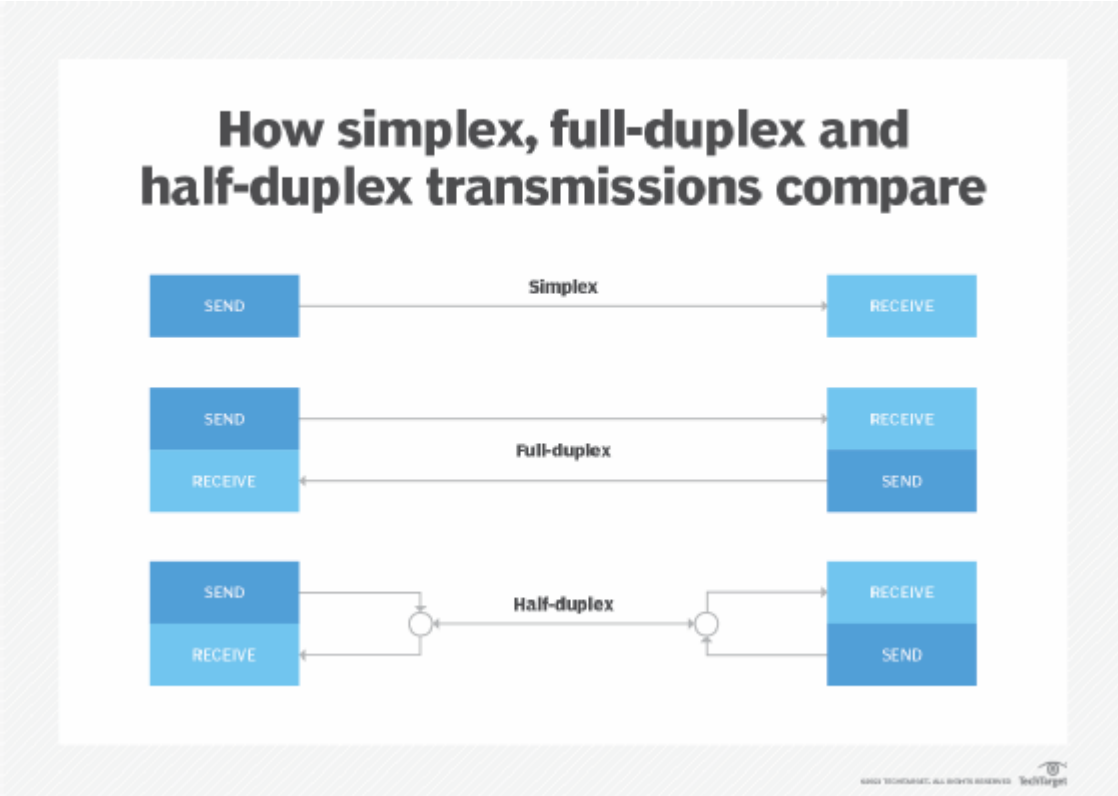
SCIs also enable serial communications with another microprocessor or with an external network. Motorola coined the term *serial communications interface* in the 1970s. In some applications, it is known as a *universal asynchronous receiver/transmitter*, or UART.

With a serial communications interface, data travels between a sender and receiver in one direction, one bit at a time.

How does an SCI work?

An SCI contains a parallel-to-serial converter that serves as a data transmitter and a serial-to-parallel converter that serves as a data receiver. The two devices are clocked separately and use independent enable and interrupt signals.

They can function in half-duplex communication mode, using only the receiver or only the transmitter, or in full-duplex mode, using the receiver and the transmitter simultaneously. The data speed is programmable.



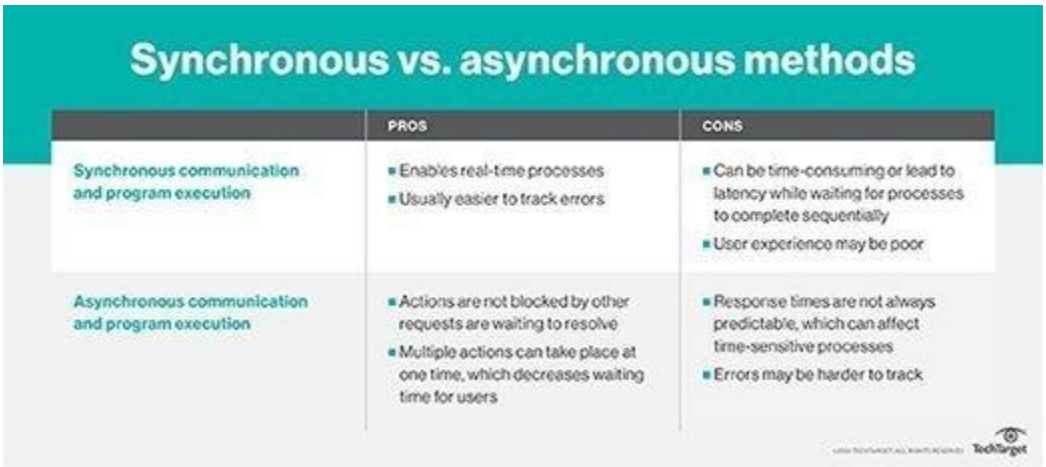
Serial communications interfaces can function in either half-duplex or full-duplex mode.

Serial vs. parallel interfaces

Serial interfaces have some advantages over parallel communications interfaces. The most significant is their simpler wiring. In addition, serial interface cables can be longer than parallel interface cables because there is less interaction, or crosstalk, among the conductors in the cable.

The term SCI is sometimes used in reference to a serial port. This is a connector found on most PCs that's intended for use with serial peripheral devices.

Special bits of data, such as a start bits and stop bits, are used to manage serial communications before data transfer occurs and after it has concluded. Other bits are used to transmit data and to receive data. Data transmission is largely asynchronous serial communication, in that it occurs only one message at a time, as opposed to synchronous transmission, where multiple data transfers can occur simultaneously in both directions.



Data transmission using a serial communications interface is mostly asynchronous.

Pros and cons of serial communications interfaces

Advantages of using serial data communications interfaces include the following:

- Have fewer wires.
- Cost less than other devices.
- Support long-distance communication between devices.

There are drawbacks to using SCIs:

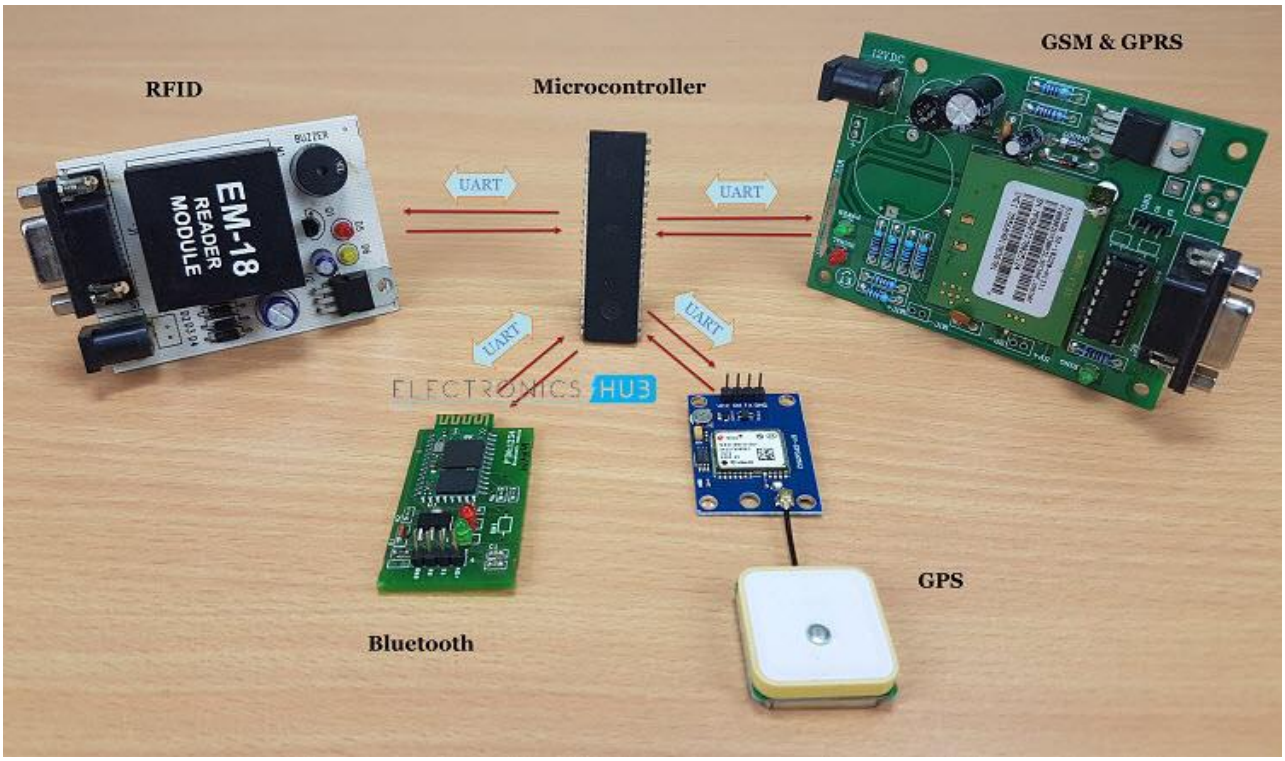
- Require data to be sent to be well organized.
- Need device controllers to manage all data communications with peripherals and embedded systems.

Serial Interface v/s Parallel Interface

Sr. No.	Parameters	Serial Interface	Parallel Interface
1.	**Main Concept**	One bit at a time.	Multiple bits at a time.
2.	**Components**	Serial interface consists of an I2C bus, SPI bus, or synchronous serial control and data lines.	Parallel interface consists of 8 data pins and 3 control lines.
3.	**Cost**	It requires fewer pins and is easy to set up hence the cost is comparatively less.	It requires more pins and is not easy to set up hence the cost is comparatively high.
4.	**Purpose**	Parallel is an external interface used for connecting computer peripherals such as printers, or any other device that requires relatively high bandwidth, to a PC.	Serial interface used for connecting serial lines to facilitate serial communication.
5.	**Transmission**	In serial interface bits of data are sent one at a time in a single stream of 1s and 0s along a single wire.	Parallel interface can move a set of 8 bits (1s and 0s) of data in parallel at the same time along 8 separate wires meaning multiple bits of data are sent simultaneously.
6.	**Speed**	Transmission speed is Low.	Transmission speed is High.
7.	**Number of Wires**	Less number of wires are used in serial interface.	More wires are used in parallel interface.
8.	**Data Sending Mechanism**	A serial interface sends data bit by bit after sending a bit at a time.	A parallel interface sends data by sending multiple bits in a parallel fashion.
9.	**Capability**	A serial interface can transmit a single stream of data at a time.	A parallel interface can transmit multiple data streams at a time.
10.	**Port Type**	A serial interface uses Male ports.	A parallel interface uses Female ports.
11.	**Redundancy**	Bottom-Up model is better suited as it ensures minimum data redundancy and focus is on re-usability.	Top-down model has a high ratio of redundancy as the size of the project increases.

Basics of UART Communication

UART or Universal Asynchronous Receiver Transmitter is a dedicated hardware associated with serial communication. It is one of the most simple and most commonly used Serial Communication techniques. Today, UART is being used in many applications like GPS Receivers, Bluetooth Modules, GSM and GPRS Modems, Wireless Communication Systems, RFID based applications etc.



If you remember older computer systems, devices like Mouse, Printer and Modem are connected using heavy connectors at the back. All these devices communicated using UART.

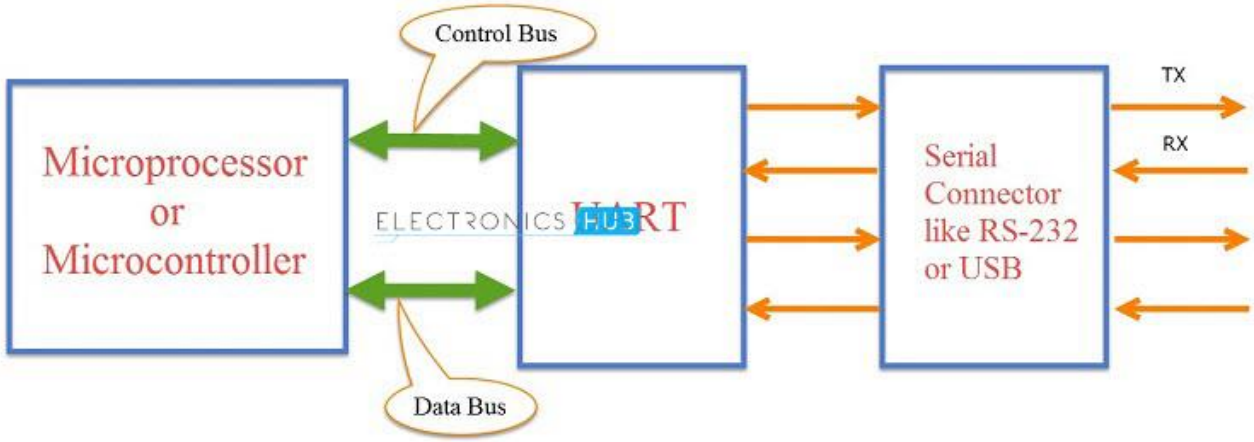
Even though USB has replaced all types of communication standards on computers and other devices, UART is still being used in the above mentioned applications.

Almost all microcontrollers/microprocessors have dedicated UART hardware built in to their architecture. The main reason for integrating the UART hardware in to

microcontrollers is that it is a serial communication and requires only two wires for communication.

UART or Universal Asynchronous Receiver Transmitter is a serial communication device that performs parallel – to – serial data conversion at the transmitter side and serial – to – parallel data conversion at the receiver side. It is universal because the parameters like transfer speed, data speed, etc. are configurable.

As mentioned in the introduction section, UART is a piece of hardware that acts as a bridge between the processor and the serial communication protocol or port. The following image shows this interface briefly. The serial communication can be anything like USB, RS – 232, etc.

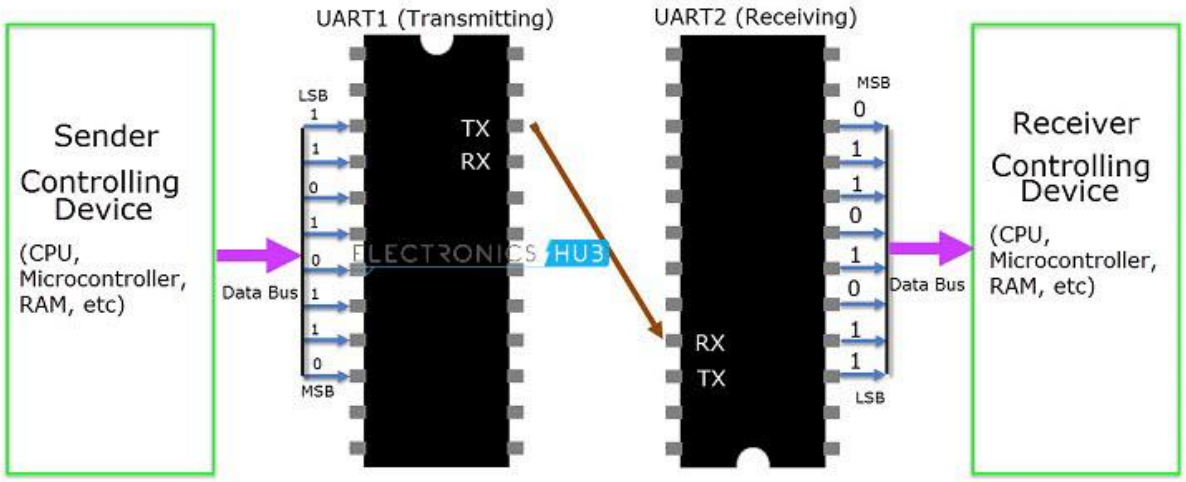


The letter ‘A’ in UART stands for Asynchronous i.e. there is no clock signal to synchronize or validate the data transmitted from transmitter and received by the receiver (Asynchronous Serial Communication).

In UART based Serial Communication, the transmitter and receiver communicate in the following manner. The UART on the sender device i.e. the transmitting UART receives parallel data from the CPU (microprocessor or microcontroller) and converts it in to serial data.

This serial data is transmitted to the UART on the receiver device i.e. receiving UART. The receiving UART, upon receiving the serial data, converts it back to parallel data and gives it to the CPU.

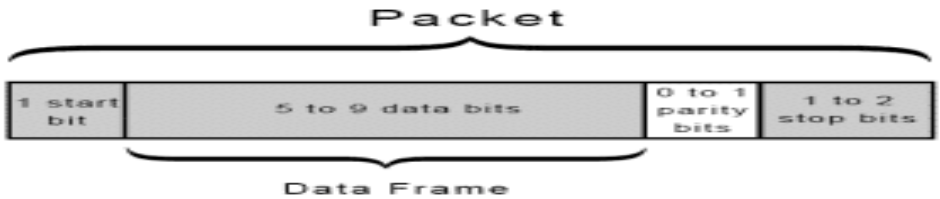
HOW UART WORKS



The image above shows a typical UART connection. The transmitting UART receives data from the controlling device through the data bus. The controlling device can be anything like a CPU of a microprocessor or a microcontroller, memory unit like a RAM or ROM, etc. The data received by the transmitting UART from the data bus is parallel data.

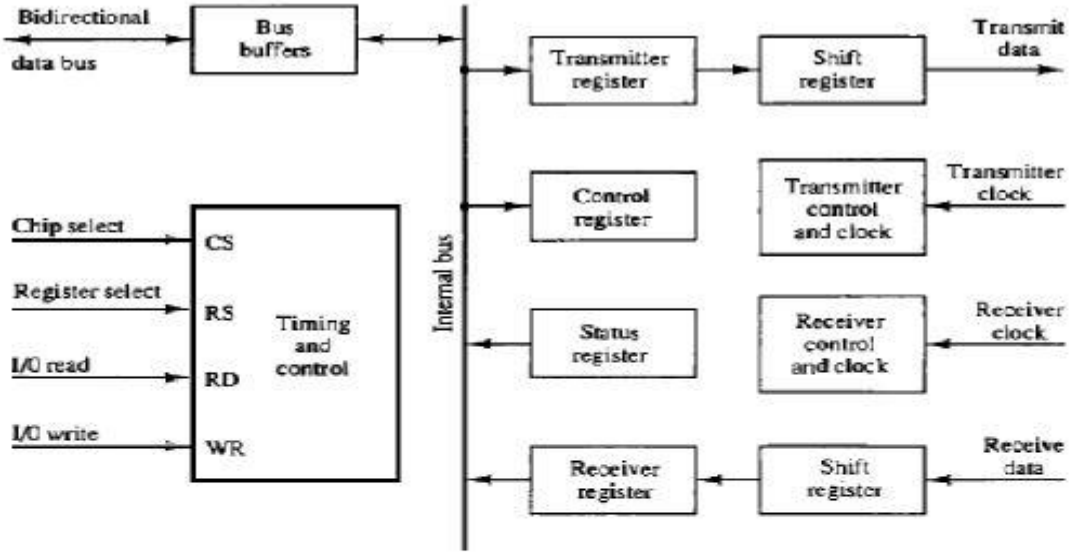
To this data, the UART adds Start, Parity and Stop bits in order to convert it into a data packet. The data packet is then converted from parallel to serial with the help of shift register and is transmitted bit – by – bit from the TX pin. The receiving UART receives this serial data at the RX pin and detects the actual data by identifying the start and stop bits. Parity bit is used to check the integrity of the data. Upon separating the start, parity and stop bits from the data packet, the data is converted to parallel data with the help of shift register. This parallel data is sent to the controller at the receiving end through a data bus.

UART transmitted data is organized into *packets*. Each packet contains 1 start bit, 5 to 9 data bits (depending on the UART), an optional *parity* bit, and 1 or 2 stop bits:



Block Diagram of UART

The block diagram of an asynchronous communication interface(UART) is shown in Fig.



CS	RS	Operation	Register selected
0	×	×	None: data bus in high-impedance
1	0	WR	Transmitter register
1	1	WR	Control register
1	0	RD	Receiver register
1	1	RD	Status register

It functions as both a transmitter and a receiver. The interface is initialized for a particular mode¬ of transfer by means of a control byte that is loaded into its control register. The transmitter register accepts a data byte from the CPU through the data bus. This byte is transferred to a shift register for serial transmission. The receiver portion receives serial information into another shift register, and when a complete data byte is accumulated, it is transferred to the receiver register. The CPU can select the receiver register to read the byte through the data bus. The bits in the status register are used for input and output

flags and for recording certain errors¬ that may occur during the transmission. The CPU can read the status register to check the status of the flag bits and to determine if any errors have occurred. The chip select and the read and write control lines communicate with the CPU. The chip select (CS) input is used to select the interface through the address bus. The register select (RS) is associated with the read (RD) and write (WR) controls. Two registers are write-only and two are read-only. The register selected is a function of the RS value and the RD and WR status, as listed in the table accompanying the diagram.

The operation of the transmitter portion of the interface is as follows. The CPU reads the status register and checks the flag to see if the transmitter register is empty. If it is empty, the CPU transfers a character to the transmitter register and the interface clears the flag to mark the register full. The first bit in the transmitter shift register is set to 0 to generate a start bit. The character is transferred in parallel from the transmitter register to the shift register and the appropriate number of stop bits is appended into the shift register. The transmitter register is then marked empty. The character can now be transmitted one bit at a time by shifting the data in the shift register at the specified baud rate.

The operation of the receiver portion of the interface is similar. The receive data input is in the 1-state when the line is idle. The receiver control monitors the receive-data line for a 0 signal to detect the occurrence of a start bit. Once a start bit has been detected, the incoming bits of the character are shifted into the shift register at the prescribed baud rate. After receiving the data bits, the interface checks for the parity and stop bits. The character without the start and stop bits is then transferred in parallel from the shift register to the receiver register. The flag in the status register is set to indicate that the receiver register is full. The CPU reads the status register and checks the flag, and if set, it reads the data from the receiver register. The interface checks for any possible errors during transmission and sets appropriate bits in the status register. The CPU can read the status register at any time to check if any errors have occurred.

USB and HDMI Standards :

HDMI and **USB** are two popular interfaces used for connecting devices and peripherals with computers and other electronic devices. The fundamental difference between HDMI and USB is that USB (Universal Serial Bus) is an interface designed for data transfer, while HDMI (High-Definition Multimedia Interface) is designed for transferring high-quality audio and video signals.

In this article, we will discuss the important differences between HDMI and USB. But before discussing the differences, let us first get an overview of HDMI and USB individually.

What is HDMI?

HDMI is an acronym for High-Definition Multimedia Interface. HDMI is a digital interface designed to transfer high-quality video and audio signals from a source device like DVD player to a display device like a TV.

In modern electronic devices, HDMI is being used as a standard display interface. HDMI were developed by a group of companies (Hitachi, Philips, Panasonic, Sony, Silicon Image, Thomson, and Toshiba) in 2002. It is a modern

digital interface that is capable of supporting both analog and digital signals. Therefore, it is used for transmitting high-definition video signals, multichannel audio signals, control signals, Ethernet signals and many more through a single cable.

HDMI connector also has a compact design making it suitable to use in small devices like laptops, tablets, etc. HDMI has a small rectangular shape connector with 19 pins.

HDMI has become a standard display interface for computers, laptops, TVs, gaming devices, etc. The more compact versions of HDMI, such as Mini-HDMI and Micro-HDMI are also being used in portable electronic devices like smartphones, tablets, PDAs, etc. HDMI offers higher data transfer rates and supports higher resolutions. It also supports HDR (High Dynamic Range) video outputs. Over a period of time, several versions of HDMI have been developed, such as HDMI 1.4, HDMI 2.0, HDMI 2.1, etc.

What is USB?

USB is an acronym for **Universal Serial Bus**. USB is an interface developed in 1996 to connect peripheral devices to computers and other electronic devices. USB is a plug and play interface, hence devices can be connected or removed without need of rebooting the computer. Therefore, it is widely used for connecting peripheral devices like keyboard, mouse, camera, hard disk drives, printers, scanners, etc. to a computer system.

USB has become a standard interface due its several advantages, such as high-speed data transfer, plug and play feature, compact size, etc.

The development of USB has eliminated the complexity of several connector types required for each peripheral. It also provides error free data transfer between devices. However, USB supports only peer-to-peer communication, which means only a single message can be transferred between the host device and the peripheral device.

After getting insights into the basics of HDMI and USB, let us now discuss their important differences.

Difference between USB and HDMI

The following table highlights all the significant differences between USB and HDMI:

Parameter	HDMI	USB
Full form	HDMI is an acronym for High-Definition Multimedia Interface.	USB is an acronym for Universal Serial Bus.
Main objective	HDMI is mainly designed for transferring high-quality audio and video signals.	USB is mainly designed to transfer data between a computer and peripheral devices.
Speed	HDMI transfers signals at a faster speed than USB.	USB transfers data at a slower speed.
Resolution	HDMI supports higher resolutions.	USB supports lower audio and video resolutions than HDMI.
Compatibility	HDMI is primarily compatible with display devices like TVs, monitors, projectors, etc.	USB is compatible with a wide range of devices.
Maximum data transfer speed	HDMI 2.1 provides a maximum data transfer speed of 48 GB/s.	USB 3.2 Gen 2 × 2 provides a maximum data transfer speed of 20 GB/s.
Cost	HDMI is more costly than USB.	USB is relatively less expensive.
Connector	HDMI connector has a larger rectangular shape and specialized design.	USB connector smaller size and less specialized design.
Power	HDMI can provide power to connected	USB can provide power to connected

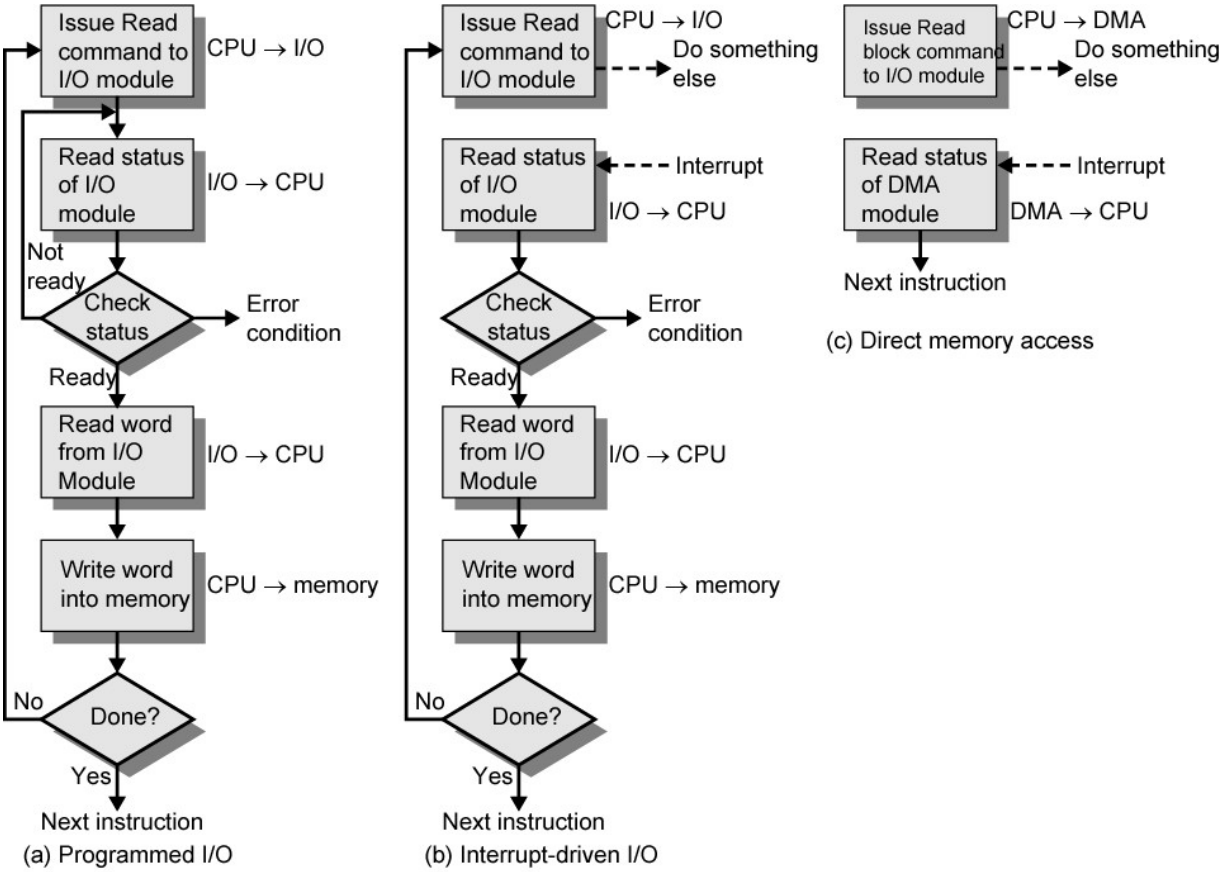
	devices.	devices.
Bandwidth	HDMI support wider bandwidth.	USB supports lower bandwidth than HDMI.
Types	HDMI is of several types like Standard HDMI, Mini HDMI, and Micro HDMI.	USB also has several types, such as Type-A USB, Type-B USB, and Type-C USB.
Data encryption	HDMI supports HDCP (High-Bandwidth Digital Content Protection) protocol for data encryption.	USB does not support any encryption protocol.
Standard cable length	HDMI cables are available in a maximum length of 15 meters.	USB cables are available in a maximum length of 5 meters.
Applications	HDMI cables are used for connecting display devices like TVs, monitors, projectors, etc.	USB cables are used for connecting peripherals, such as keyboard, mouse, printer, HDD, etc.

Conclusion

Both USB and HDMI are widely used communication interfaces used in almost all modern electronic devices. But they are absolutely different from each other as described in the above table. The most significant difference between HDMI and USB is that HDMI (High-Definition Multimedia Interface) is designed to transfer high-quality audio and video signals, while USB (Universal Serial Bus) is designed to transfer signals between computer and peripherals.

I/O Techniques

- Programmed
- Interrupt driven
- Direct Memory Access (DMA)



Programmed I/O

- Programmed I/O operations are the result of I/O instructions written in the computer program.
- In programmed I/O, each data transfer in initiated by the instructions in the CPU and hence the CPU is in the continuous monitoring of the interface.
- Input instruction is used to transfer data from I/O device to CPU, store instruction is used to transfer data from CPU to memory and output instruction is used to transfer data from CPU to I/O device.
- This technique is generally used in very slow speed computer and is not a efficient method if the speed of the CPU and I/O is different.

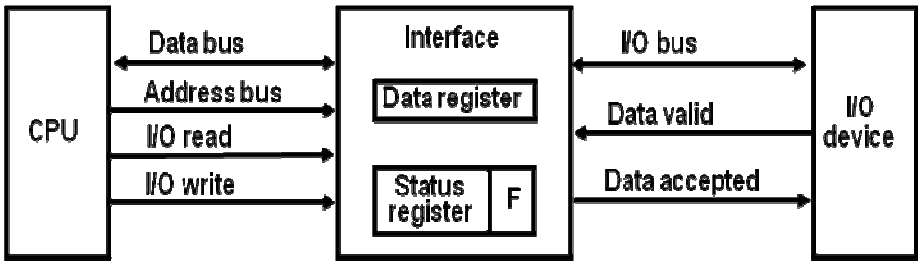


Fig: Data transfer from I/O device to CPU

- I/O device places the data on the I/O bus and enables its data valid signal
- The interface accepts the data in the data register and sets the F bit of status register and also enables the data accepted signal.
- Data valid line is disables by I/O device.
- CPU is in a continuous monitoring of the interface in which it checks the F bit of the status register.
    - If it is set i.e. 1, then the CPU reads the data from data register and sets F bit to zero
    - If it is reset i.e. 0, then the CPU remains monitoring the interface.
- Interface disables the data accepted signal and the system goes to initial state where next item of data is placed on the data bus.

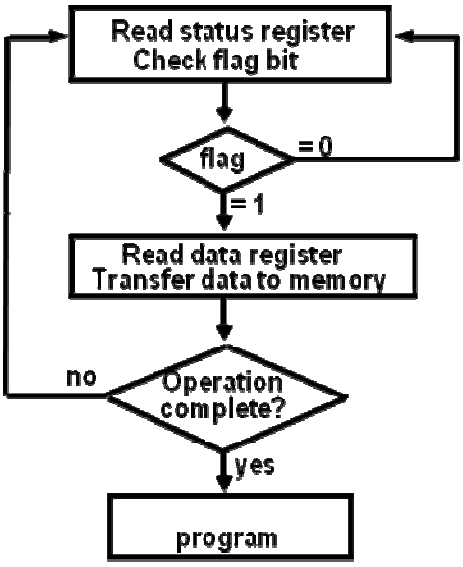


Fig: Flowchart for CPU program to input data

Characteristics:

- Continuous CPU involvement
- CPU slowed down to I/O speed
- Simple
- Least hardware

Polling, or polled operation, in computer science, refers to actively sampling the status of an external device by a client program as a synchronous activity. Polling is most often used in terms of input/output (I/O), and is also referred to as **polled I/O or software driven I/O.**

Interrupt-driven I/O

- Polling takes valuable CPU time
- Open communication only when some data has to be passed -> *Interrupt*.
- I/O interface, instead of the CPU, monitors the I/O device
- When the interface determines that the I/O device is ready for data transfer, it generates an *Interrupt Request* to the CPU
- Upon detecting an interrupt, CPU stops momentarily the task it is doing, branches to the service routine to process the data transfer, and then returns to the task it was performing

The problem with programmed I/O is that the processor has to wait a long time for the I/O module of concern to be ready for either reception or transmission of data. The processor, while waiting, must repeatedly interrogate the status of the I/O module. As a result, the level of the performance of the entire system is severely degraded. An alternative is for the processor to issue an I/O command to a module and then go on to do some other useful work. The I/O module will then interrupt the processor to request service when it is ready to exchange data with processor. The processor then executes the data transfer, and then resumes its former processing. The interrupt can be initiated either by software or by hardware.

Interrupt Driven I/O basic operation

- CPU issues read command
- I/O module gets data from peripheral whilst CPU does other work
- I/O module interrupts CPU
- CPU requests data
- I/O module transfers data

Interrupt Processing from CPU viewpoint

- Issue read command
- Do other work
- Check for interrupt at end of each instruction cycle
- If interrupted:-
    - Save context (registers)
    - Process interrupt
    - Fetch data & store

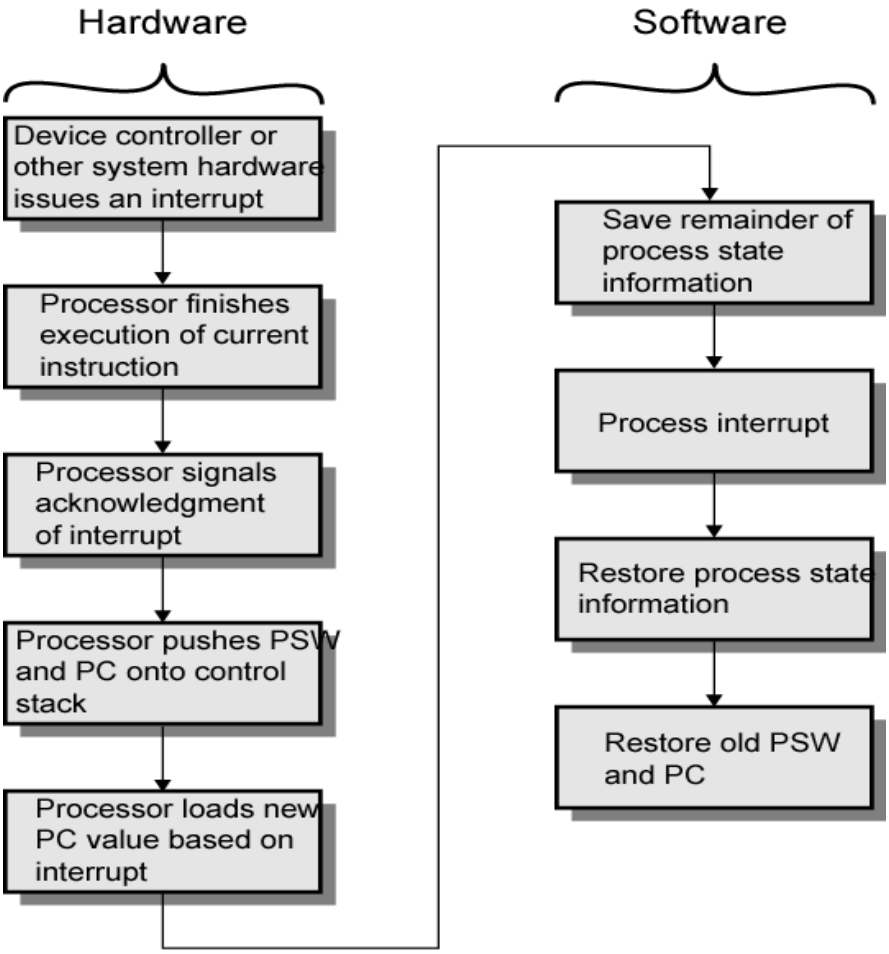


Fig: Simple Interrupt Processing

Priority Interrupt

- Determines which interrupt is to be served first when two or more requests are made simultaneously
- Also determines which interrupts are permitted to interrupt the computer while another is being serviced
- Higher priority interrupts can make requests while servicing a lower priority interrupt

Priority Interrupt by Software (Polling)

- Priority is established by the order of polling the devices (interrupt sources), that is identify the highest-priority source by software means
- One common branch address is used for all interrupts
- Program polls the interrupt sources in sequence
- The highest-priority source is tested first
- Flexible since it is established by software
- Low cost since it needs a very little hardware
- Very slow

Priority Interrupt by Hardware

- Require a priority interrupt manager which accepts all the interrupt requests to determine the highest priority request
- Fast since identification of the highest priority interrupt request is identified by the hardware
- Fast since each interrupt source has its own interrupt vector to access directly to its own service routine

1. Daisy Chain Priority (Serial)

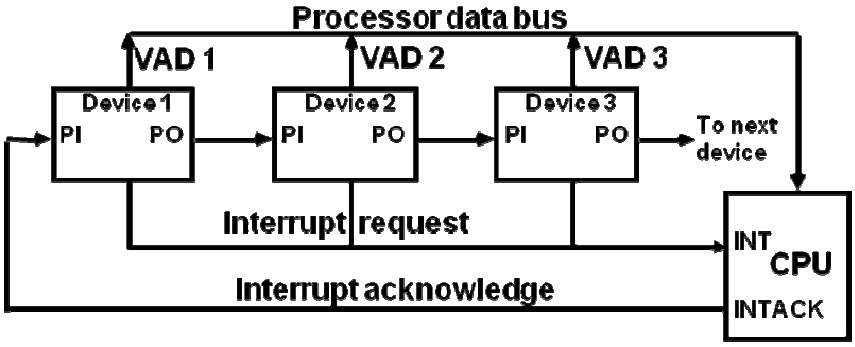


Fig: Daisy Chain priority Interrupt

- Interrupt Request from any device
- CPU responds by INTACK
- Any device receives signal(INTACK) at PI puts the VAD on the bus
- Among interrupt requesting devices the only device which is physically closest to CPU gets INTACK and it blocks INTACK to propagate to the next device

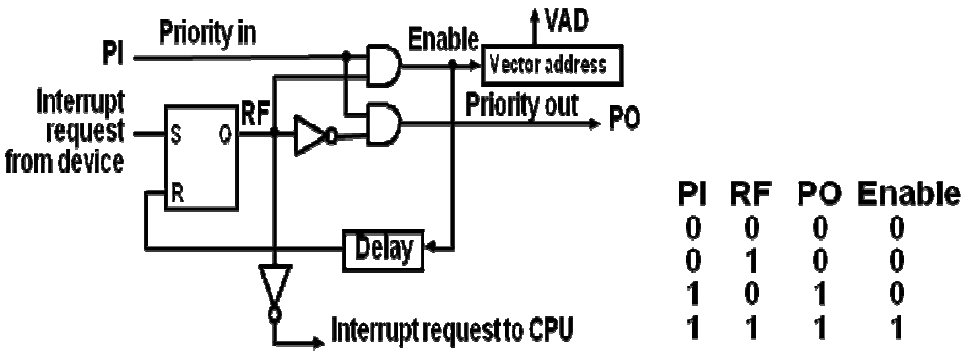


Fig: One stage of Daisy chain priority arrangement

2. Parallel Priority

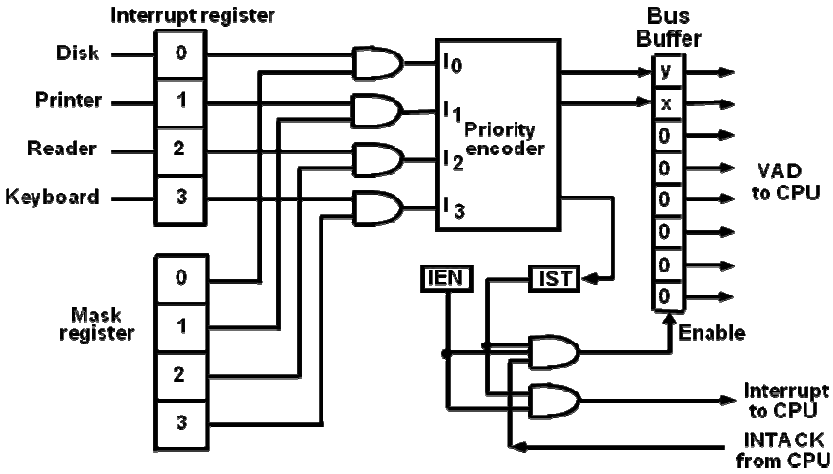


Fig: Parallel priority interrupts hardware

- IEN: Set or Clear by instructions ION or IOF
- IST: Represents an unmasked interrupt has occurred. INTACK enables tristate Bus Buffer to load VAD generated by the Priority Logic
- Interrupt Register:
  - Each bit is associated with an Interrupt Request from different Interrupt Source - different priority level
  - Each bit can be cleared by a program instruction
- Mask Register:
  - Mask Register is associated with Interrupt Register
  - Each bit can be set or cleared by an Instruction

Priority Encoder

- Determines the highest priority interrupt when more than one interrupts take place

Inputs				Outputs			
I_0	I_1	I_2	I_3	x	y	IST	Boolean functions
1	d	d	d	0	0	1	
0	1	d	d	0	1	1	
0	0	1	d	1	0	1	$x = I_0' \; I_1'$
0	0	0	1	1	1	1	$y = I_0' I_1 + I_0' I_2'$
0	0	0	0	d	d	0	$(IST) = I_0 + I_1 + I_2 + I_3$

Fig: Priority Encoder Truth Table

Interrupt Cycle

At the end of each Instruction cycle

- CPU checks IEN and IST
- If IEN and IST = 1, CPU -> Interrupt Cycle
    - SP ← SP – 1; Decrement stack pointer
    - M[SP] ← PC; Push PC into stack
    - INTACK ← 1; Enable interrupt acknowledge
    - PC ← VAD; Transfer vector address to PC
    - IEN ← 0; Disable further interrupts
    - Go To Fetch to execute the first instruction in the interrupt service routine

Direct Memory access

- Large blocks of data transferred at a high speed to or from high speed devices, magnetic drums, disks, tapes, etc.
- DMA controller Interface that provides I/O transfer of data directly to and from the memory and the I/O device
- CPU initializes the DMA controller by sending a memory address and the number of words to be transferred
- Actual transfer of data is done directly between the device and memory through DMA controller -> Freeing CPU for other tasks

The transfer of data between the peripheral and memory without the interaction of CPU and letting the peripheral device manage the memory bus directly is termed as Direct Memory Access (DMA).

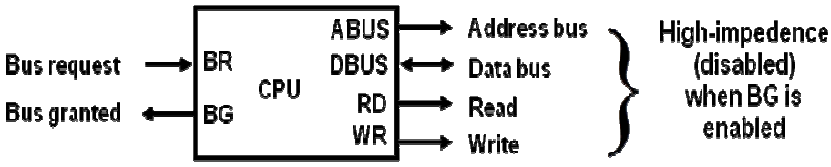


Fig: CPU bus signal for DMA transfer

The two control signals Bus Request and Bus Grant are used to fascinate the DMA transfer. The bus request input is used by the DMA controller to request the CPU for the control of the buses. When BR signal is high, the CPU terminates the execution of the current instructions and then places the address, data, read and write lines to the high impedance state and sends the bus grant signal. The DMA controller now takes the control of the buses and transfers the data directly between memory and I/O without processor interaction. When the transfer is completed, the bus request signal is made low by DMA. In response to which CPU disables the bus grant and again CPU takes the control of address, data, read and write lines.

The transfer of data between the memory and I/O of course facilitates in two ways which are DMA Burst and Cycle Stealing.

DMA Burst: The block of data consisting a number of memory words is transferred at a time.

Cycle Stealing: DMA transfers one data word at a time after which it must return control of the buses to the CPU.

- CPU is usually much faster than I/O (DMA), thus CPU uses the most of the memory cycles
- DMA Controller steals the memory cycles from CPU
- For those stolen cycles, CPU remains idle
- For those slow CPU, DMA Controller may steal most of the memory cycles which may cause CPU remain idle long time

DMA Controller

The DMA controller communicates with the CPU through the data bus and control lines. DMA select signal is used for selecting the controller, the register select is for selecting the register. When the bus grant signal is zero, the CPU communicates through the data bus to read or write into the DMA register. When bus grant is one, the DMA controller takes the control of buses and transfers the data between the memory and I/O.

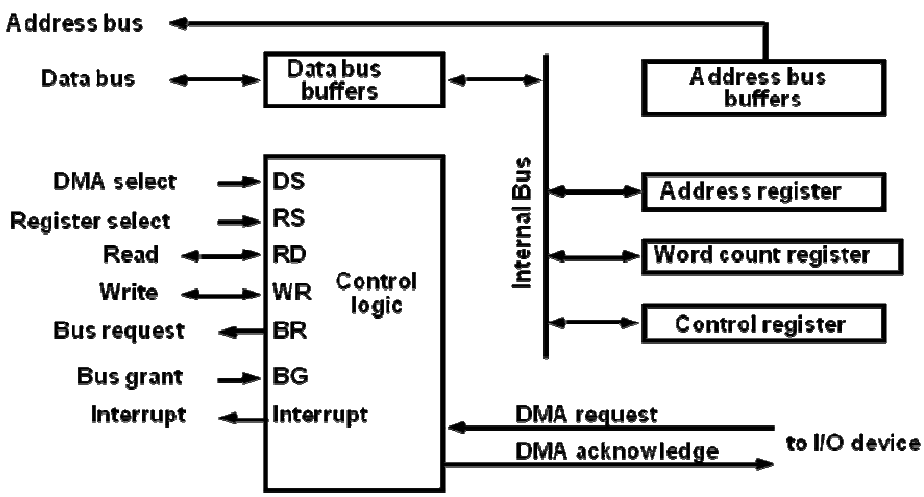


Fig: Block diagram of DMA controller

The address register specifies the desired location of the memory which is incremented after each word is transferred to the memory. The word count register holds the number of words to be transferred which is decremented after each transfer until it is zero. When it is zero, it indicates the end of transfer. After which the bus grant signal from CPU is made low and CPU returns to its normal operation. The control register specifies the mode of transfer which is Read or Write.

DMA Transfer

- DMA request signal is given from I/O device to DMA controller.

- DMA sends the bus request signal to CPU in response to which CPU disables its current instructions and initialize the DMA by sending the following information.
    - The starting address of the memory block where the data are available (for read) and where data to be stored (for write)
    - The word count which is the number of words in the memory block
    - Control to specify the mode of transfer
    - Sends a bust grant as 1 so that DMA controller can take the control of the buses
    - DMA sends the DMA acknowledge signal in response to which peripheral device puts the words in the data bus (for write) or receives a word from the data bus (for read).

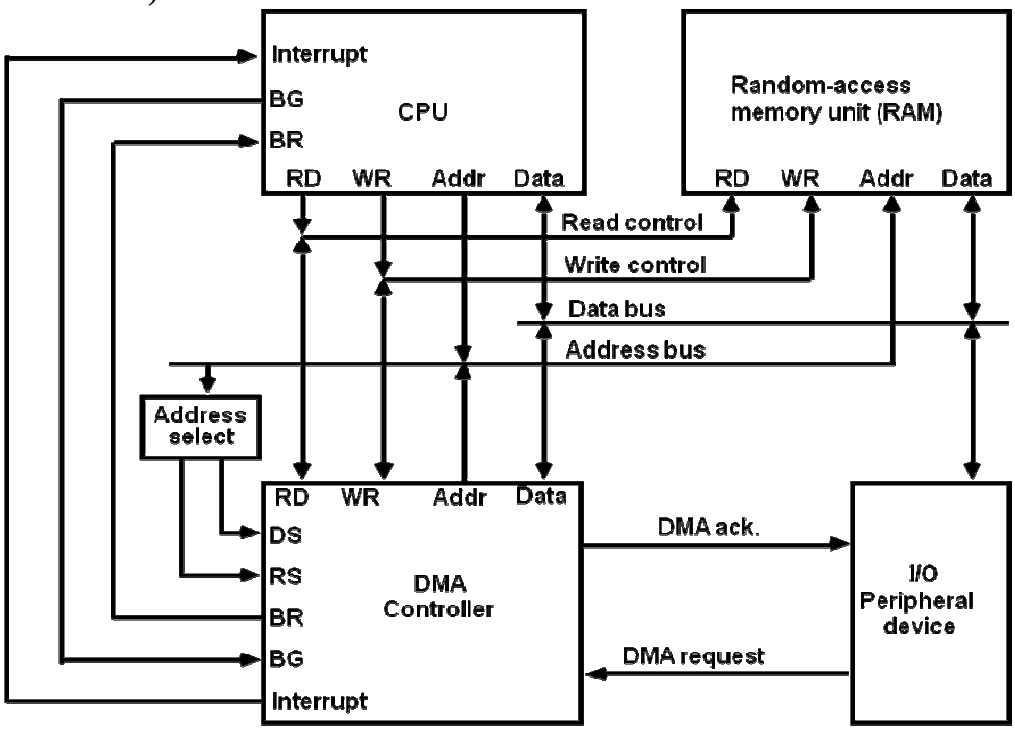


Fig: DMA transfer in a computer system

DMA Operation

- CPU tells DMA controller:-
    - Read/Write
    - Device address
    - Starting address of memory block for data
    - Amount of data to be transferred
- CPU carries on with other work
- DMA controller deals with transfer
- DMA controller sends interrupt when finished

I/O Processors

- Processor with direct memory access capability that communicates with I/O devices
- Channel accesses memory by cycle stealing

- Channel can execute a Channel Program
- Stored in the main memory
- Consists of Channel Command Word(CCW)
- Each CCW specifies the parameters needed by the channel to control the I/O devices and perform data transfer operations
- CPU initiates the channel by executing a channel I/O class instruction and once initiated, channel operates independently of the CPU

A computer may incorporate one or more external processors and assign them the task of communicating directly with the I/O devices so that no each interface need to communicate with the CPU. An I/O processor (IOP) is a processor with direct memory access capability that communicates with I/O devices. IOP instructions are specifically designed to facilitate I/O transfer. The IOP can perform other processing tasks such as arithmetic logic, branching and code translation.

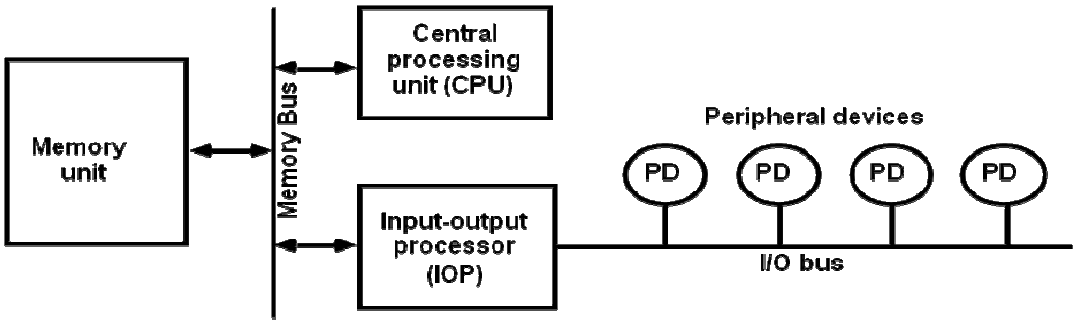


Fig: Block diagram of a computer with I/O Processor

The memory unit occupies a central position and can communicate with each processor by means of direct memory access. The CPU is responsible for processing data needed in the solution of computational tasks. The IOP provides a path for transferring data between various peripheral devices and memory unit.

In most computer systems, the CPU is the master while the IOP is a slave processor. The CPU initiates the IOP and after which the IOP operates independent of CPU and transfer data between the peripheral and memory. For example, the IOP receives 5 bytes from an input device at the device rate and bit capacity. After which the IOP packs them into one block of 40 bits and transfer them to memory. Similarly the O/P word transfer from memory to IOP is directed from the IOP to the O/P device at the device rate and bit capacity.

CPU – IOP Communication

The memory unit acts as a message center where each processor leaves information for the other. The operation of typical IOP is appreciated with the example by which the CPU and IOP communication.

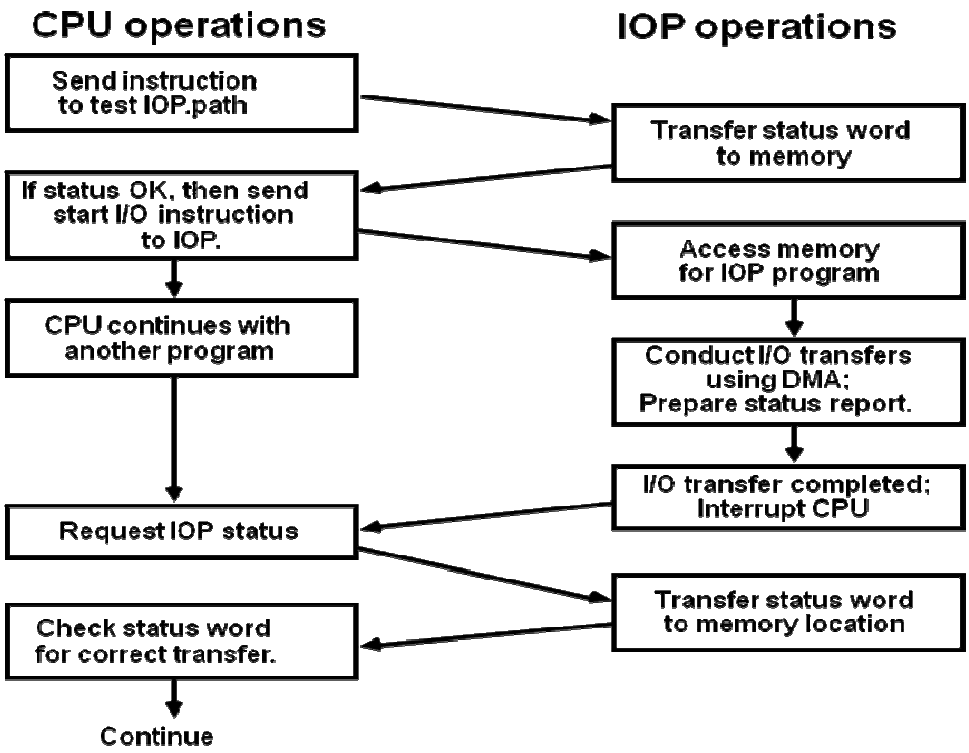


Fig: CPU – IOP communication

- The CPU sends an instruction to test the IOP path.
- The IOP responds by inserting a status word in memory for the CPU to check.
- The bits of the status word indicate the condition of the IOP and I/O device, such as IOP overload condition, device busy with another transfer or device ready for I/O transfer.
- The CPU refers to the status word in in memory to decide what to do next.
- If all right up to this, the CPU sends the instruction to start I/O transfer.
- The CPU now continues with another program while IOP is busy with I/O program.
- When IOP terminates the execution, it sends an interrupt request to CPU.
- CPU responds by issuing an instruction to read the status from the IOP.
- IOP responds by placing the contents to its status report into specified memory location.
- Status word indicates whether the transfer has been completed or with error.

Data Communication Processor

- Distributes and collects data from many remote terminals connected through telephone and other communication lines.
- Transmission:
  - Synchronous
  - Asynchronous
- Transmission Error:
  - Parity
  - Checksum
  - Cyclic Redundancy Check

    - Longitudinal Redundancy Check
- Transmission Modes:
    - Simples
    - Half Duplex
    - Full Duplex
- Data Link & Protocol

A data communication (command) processor is an I/O processor that distributes and collects data from remote terminals connected through telephone and other communication lines. In processor communication, processor communicates with the I/O device through a common bus i.e. data and control with sharing by each peripherals. In data communication, processor communicates with each terminal through a single pair of wires.

The way that remote terminals are connected to a data communication processor is via telephone lines or other public or private communication facilities. The data communication may be either through synchronous transmission or through asynchronous transmission. One of the functions of data communication processor is check for transmission errors. An error can be detected by checking the parity in each character received. The other ways are checksum, longitudinal redundancy check (LRC) and cyclic redundancy check (CRC).

Data can be transmitted between two points through three different modes. First is simplex where data can be transmitted in only one direction such as TV broadcasting. Second is half duplex where data can be transmitted in both directions at a time such as walkie-talkie. The third is full duplex where data can be transmitted in both directions simultaneously such as telephone.

The communication lines, modems and other equipment used in the transmission of information between two or more stations is called data link. The orderly transfer of information in a data link is accomplished by means of a protocol.